Chapter 6: Performance Considerations 2

CSCD439/539 GPU Programming

Performance Considerations

- Memory Coalescing
- Shared Memory Bank Conflicts
- Control-Flow Divergence
- Occupancy
- Kernel Launch Overheads

Outline Today

- Thoughts about optimizing your cuda code
- Some case studies about memory coalescing

Before You Optimize

- Always measure where your time is going!
  - Even if you think you know where it is going
  - Start coarse, go fine-grained as need be
- nvprof
  - Command line profiler for cuda applications
  - Distributed with cuda sdk
  - E.g. nvprof –s cudaAppExe

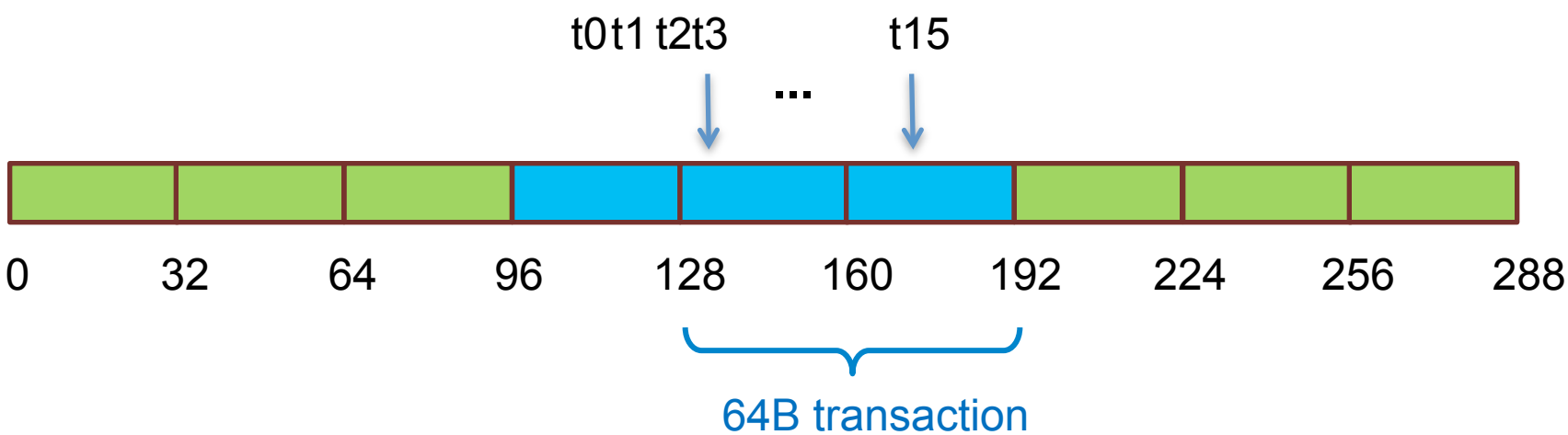
Before You Optimize

- Keep in mind Amdahl’s Law when optimizing any part of your code
  - Don’t continue to optimize once a part is only a small fraction of overall execution time.
- Amdahl’s Law,
  - Problem solution consists of two parts, parallelizable and non-parallelizable.
  - Parallel programming can only improve performance in the parallelizable part.

CASE STUDY MEMORY COALESCING

Threads 0-15 access 4-byte words at addresses 116-176

- Thread 3 is lowest active, accesses address 128
- 128-byte segment: 128-255 (reduce to 64B)



Consider the stride of your accesses

```
__global__ void foo(int* input,
                    float3* input2)
{
  int i = blockDim.x * blockIdx.x
        + threadIdx.x;
  // Stride 1
  int a = input[i];
  // Stride 2, half the bandwidth is wasted
  int b = input[2*i];
  // Stride 3, 2/3 of the bandwidth wasted
  float c = input2[i].x;
}
```

Example: Array of Structures (AoS)

```
typedef struct
{
  int key;
  int value;
  int flag;
}record;

record  *d_records;
cudaMalloc((void**)&d_records, ...);
```

Example: Structure of Arrays (SoA)

```
typedef struct SoA
{
  int   * keys;
  int   * values;
  int   * flags;
}SoA;

SoA d_SoA_data;
cudaMalloc((void**)&d_SoA_data.keys, ...);
cudaMalloc((void**)&d_SoA_data.values, ...);
cudaMalloc((void**)&d_SoA_data.flags, ...);
```

Example: SoA vs. AoS

```
__global__ void bar(record *AoS_data,
                    SoA SoA_data)
{
  int i = blockDim.x * blockIdx.x
        + threadIdx.x;
  // AoS wastes bandwidth
  int key = AoS_data[i].key;
  // SoA efficient use of bandwidth
  int key_better = SoA_data.keys[i];
}
```

Memory Coalescing

- Structure of array is often better than array of structures
  - Very clear win on regular, stride 1 access patterns
  - Unpredictable or irregular access patterns are case-by-case.

Analyzing Simple Matrix Kernel

```
__global__ void MatrixMulKernel(float* d_M, float* d_N, float* d_P, int Width) {

  // Calculate the row index of the d_Pelement and d_M
  int Row = blockIdx.y*blockDim.y+threadIdx.y;

  // Calculate the column index of d_P and d_N
  int Col = blockIdx.x*blockDim.x+threadIdx.x;

  if ((Row < Width) && (Col < Width)) {
    float Pvalue = 0;
    // each thread computes one element of the block sub-matrix
    for (int K=0 ; k < Width; ++k) {
      Pvalue += d_M[Row*Width+k]*d_N[k*Width+Col];
    }
    d_P[Row*Width+Col] = Pvalue;
  }

}
```

FIGURE 4.7

A simple matrix–matrix multiplication kernel using one thread to compute each d_P element.

Analyzing Simple Matrix Kernel

- What if the block shape is N by 1. (N rows), can you identify any memory coalescing? NO!!

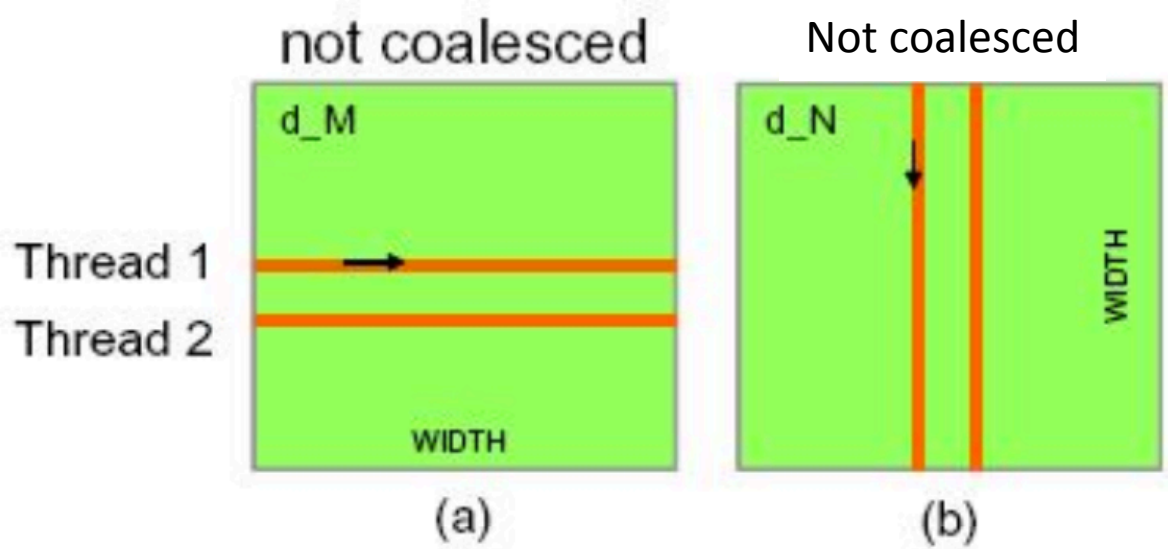


FIGURE 6.7

Memory access patterns in C 2D arrays for coalescing.

Analyzing Simple Matrix Kernel

- block shape is N by 1, Accessing Matrix M

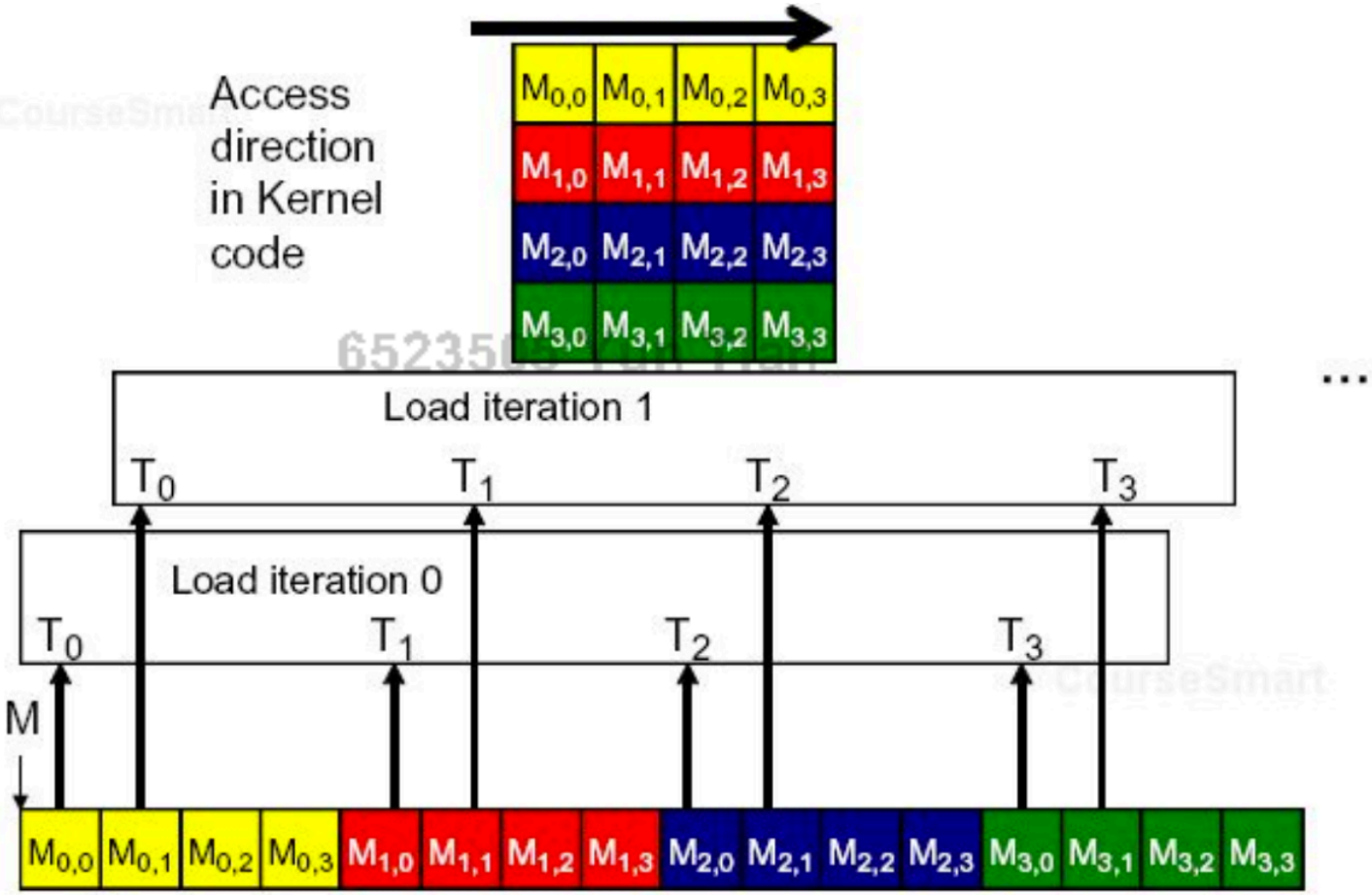


FIGURE 6.9

An uncoalesced access pattern.

Analyzing Simple Matrix Kernel

- What if the block shape is 1 by N (1 row and N cols) (or N by N), can you identify any memory coalescing? Yes!!

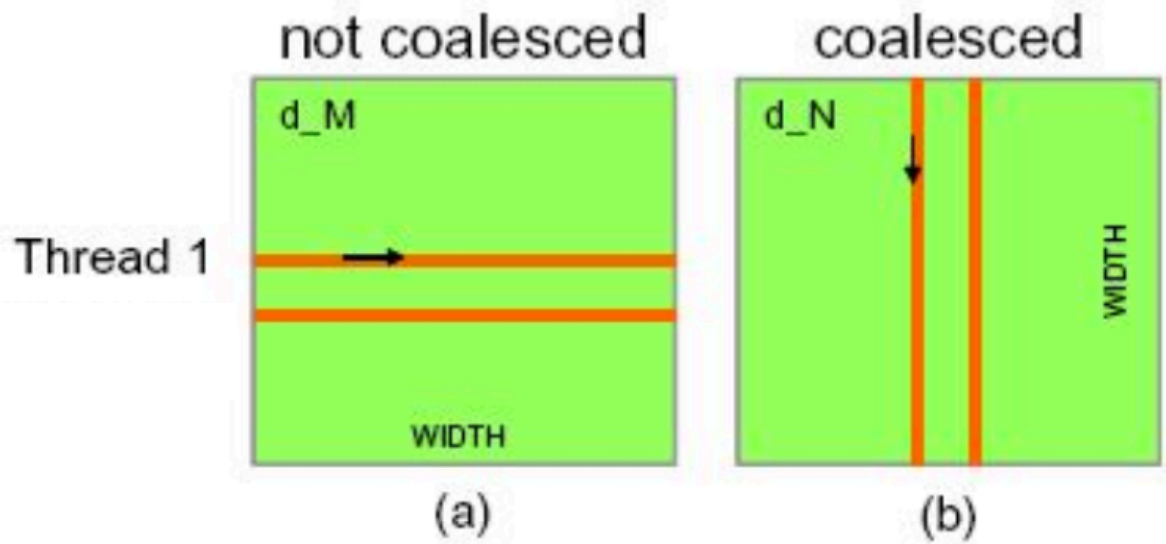


FIGURE 6.7

Memory access patterns in C 2D arrays for coalescing.

Analyzing Tiled Matrix Kernel

- If the block shape is (N by N), can you identify any memory coalescing? Yes!!
- Now, each row in global 2D matrix **covered(intersected)** by a tile is coalesced.

```
__global__ void MatrixMulKernel(float* d_M, float* d_N, float* d_P,
     int Width) {

 1.   __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
 2.   __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];

 3.   int bx = blockIdx.x;  int by = blockIdx.y;
 4.   int tx = threadIdx.x; int ty = threadIdx.y;

      // Identify the row and column of the d_P element to work on
 5.   int Row = by * TILE_WIDTH + ty;
 6.   int Col = bx * TILE_WIDTH + tx;

 7.   float Pvalue = 0;
      // Loop over the d_M and d_N tiles required to compute d_P element
 8.   for (int m = 0; m < Width/TILE_WIDTH; ++m) {

        // Coolaborative loading of d_M and d_N tiles into shared memory
 9.     Mds[ty][tx] = d_M[Row*Width + m*TILE_WIDTH + tx];
10.     Nds[ty][tx] = d_N[(m*TILE_WIDTH + ty)*Width + Col];
11.     __syncthreads();

12.     for (int k = 0; k < TILE_WIDTH; ++k) {
13.       Pvalue += Mds[ty][k] * Nds[k][tx];
        }
14.     __syncthreads();
      }
15.   d_P[Row*Width + Col] = Pvalue;
    }
```

FIGURE 5.12

Tiled matrix multiplication kernel using shared memory.

Analyzing Tiled Matrix Kernel

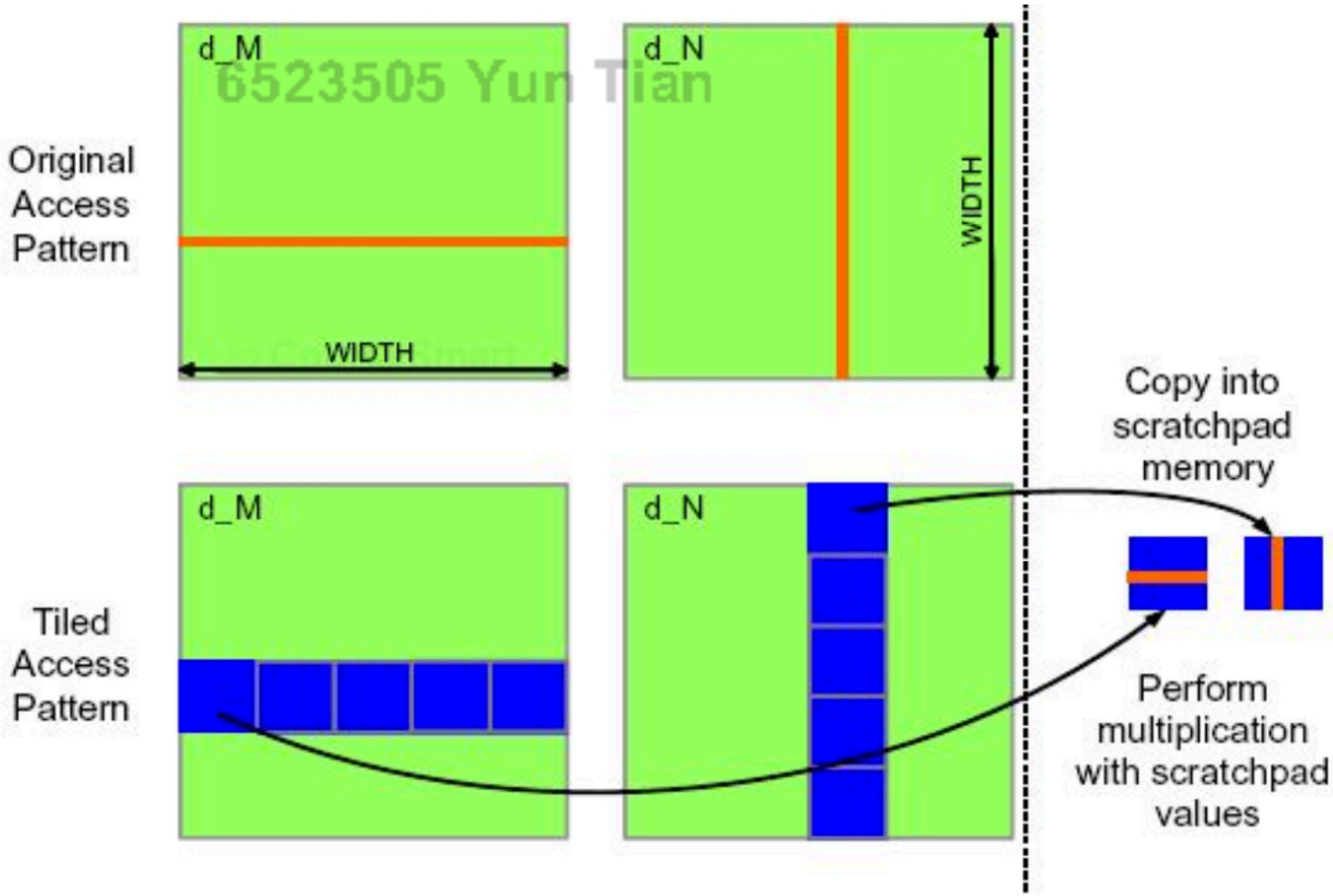


FIGURE 6.10

Using shared memory to enable coalescing.

Performance Considerations

- Control-Flow Divergence
- Occupancy
- Kernel Launch Overheads